

# Mobile Application Privacy Risk Assessments

From User-authored Scenarios

# BACKGROUND INFORMATION

- Mobile apps collect lots of personal information from users
- Some development teams lack of legal expertise and privacy resources
- Developers DO NOT KNOW which information the user REALLY CARES ABOUT
- Privacy Abuse and Legal Disputes sometimes occur if the app has been released



# TRADITIONAL SOLUTIONS



- Utilizing the permission access system in the Play Store
- Writing privacy Policy before installing the app
- Watching the response from the user



# TRADITIONAL SOLUTIONS

- Limited Permissions are covered in the Play Store
- Users do not read the Privacy Policy most of the time
- It is too late to make up for the losses caused by information leakage



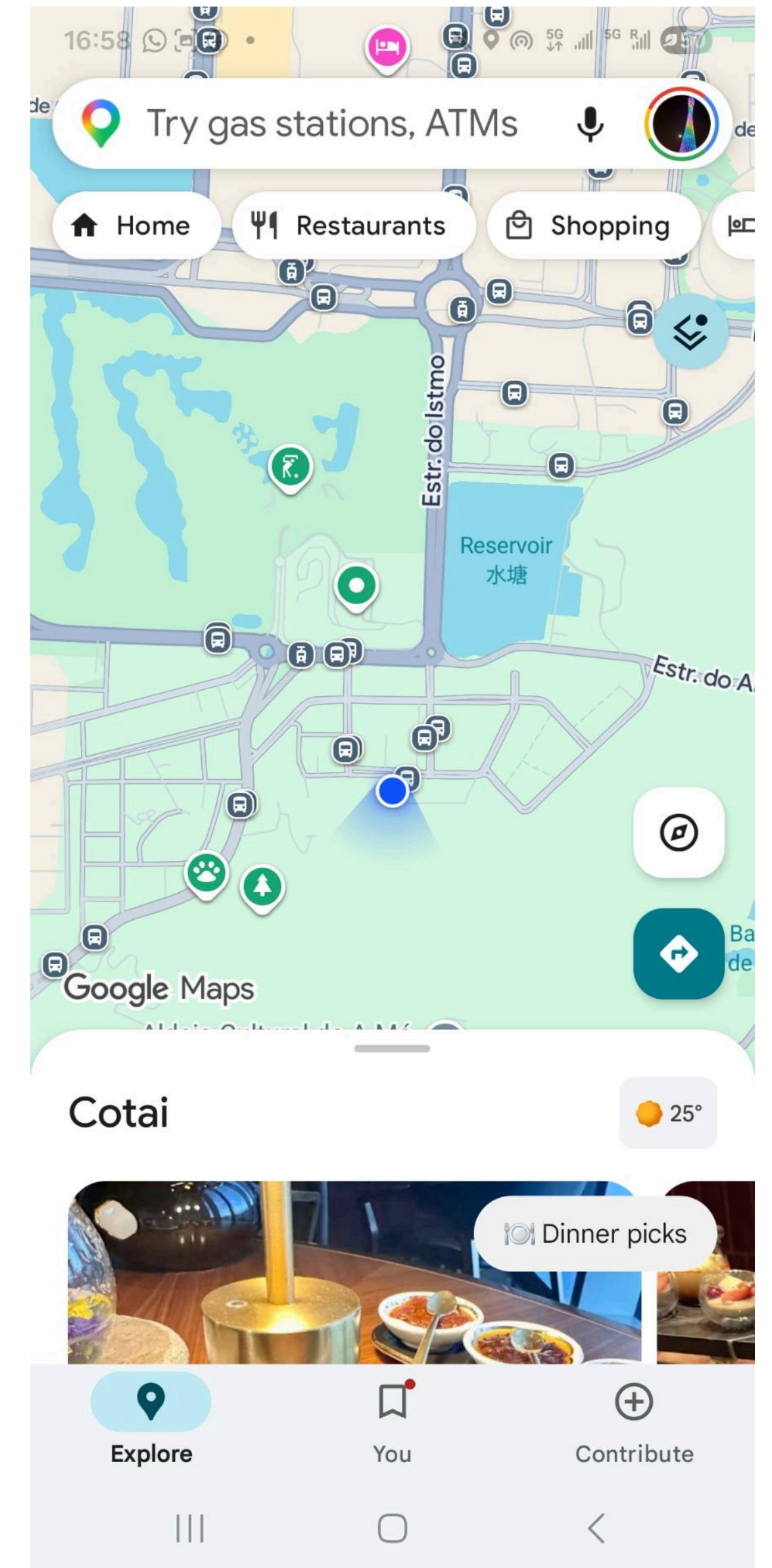
# SCENARIO-DRIVEN PRIVACY RISK ASSESSMENT

- Eliminating User-authored Scenarios
- Training the NER Models
- Eliciting Privacy Risk Scores



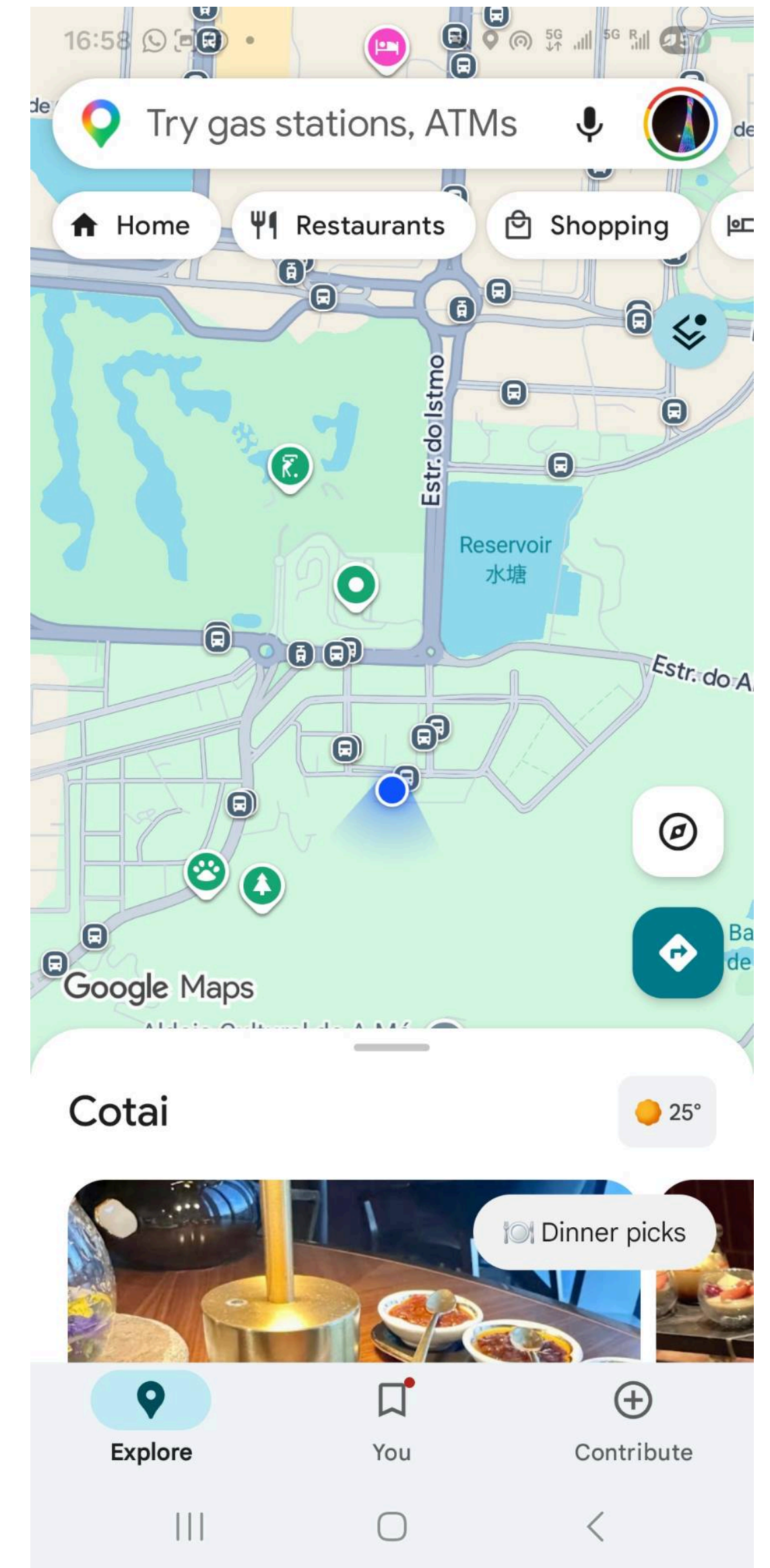
# ELIMINATING USER SCENARIOS

- Provide a screenshot for the user
- Ask the user to describe the following question with  $\geq 150$  words
- 1- What do you want to do?
- 2- How do you take action?
- 3- Which information does the application obtain?



# MODEL TRAINING

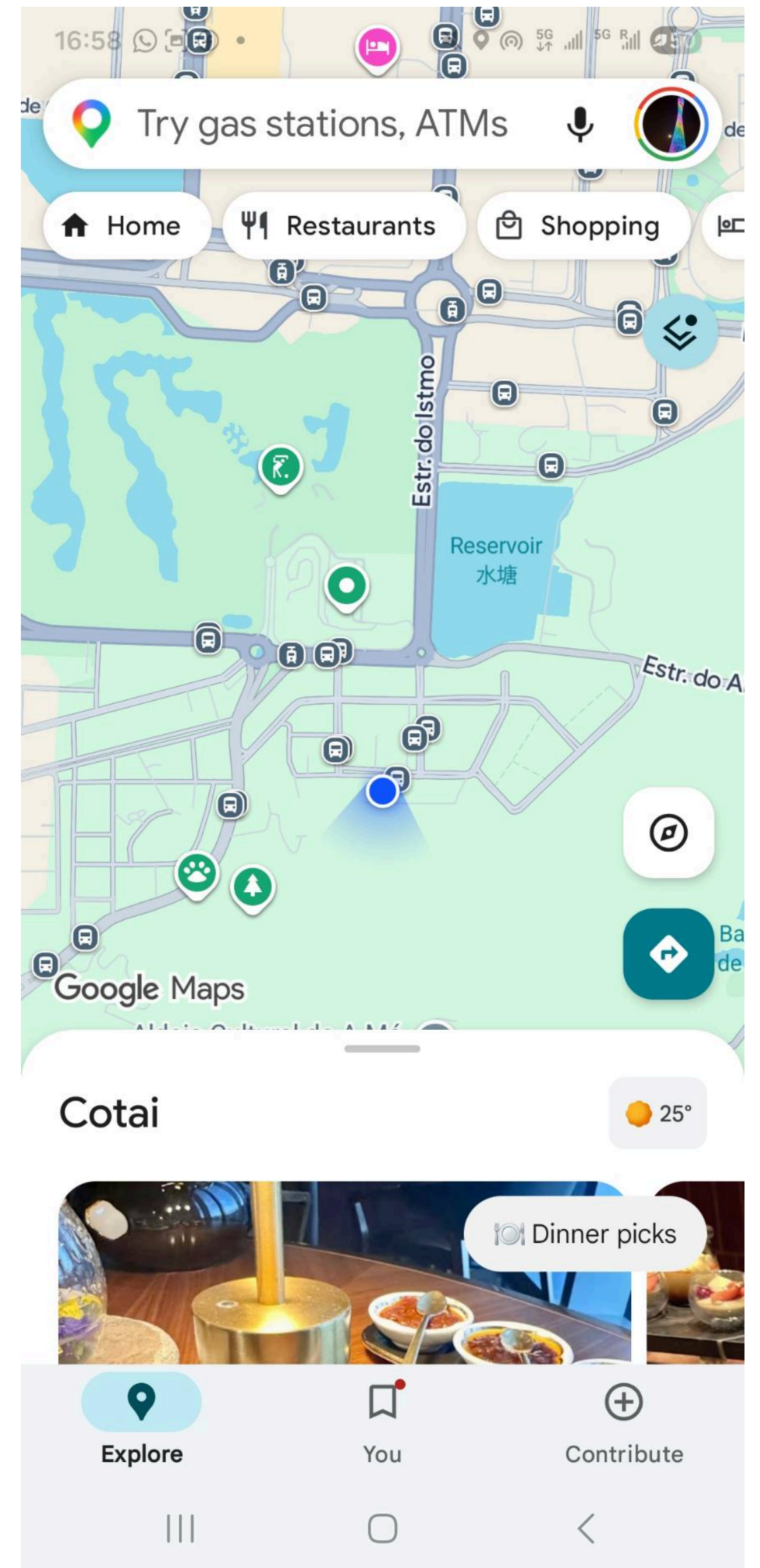
- Utilize the pre-trained NER Model for Training
- Input the Scenario Into the Model
- Identify and Label the Type of Personal Information
- Remove ALL False Positive Words (e.g., “of” “the” “my”)





# ELICITING RISK SCORES

- Ask The User For Their Willingness to Share This Information With a Third Party For Any Purpose.
- Utilize Six Options for Selection, From Very Willing to Very Unwilling





# RESULTS

- Collected 203 scenarios and 2338 user comments
- BERT has the best performance, which has 70% prediction and 73% recalls.
- The user can correctly distinguish if the information is sensitive (TEL vs. age)
- 66% of the users are “mixed.”
- e.g., I don’t mind my age and email address, but I do mind my searching records

# EXPIREMENTS

| Category        | Tool / Framework         | Purpose                                                                             |
|-----------------|--------------------------|-------------------------------------------------------------------------------------|
| Environm<br>ent | Conda<br>(Anaconda 4.12) | Dependency management and virtual environments.                                     |
| Platform        | Jupyter<br>Notebook      | Interactive development and experiment tracking via .ipynb.                         |
| Core<br>Models  | BERT + CRF               | Hybrid architecture: Transformer embeddings with a CRF layer for sequence decoding. |
| Processin<br>g  | SpaCy                    | Linguistic preprocessing, including tokenization and Part-of-Speech (POS) tagging.  |
| Evaluatio<br>n  | seqeval                  | Strict CoNLL-style metrics (Precision, Recall, F1) for span-level accuracy.         |

SCENARIO\_RISK\_SCORES

coding

samples

Interrater Reliability.ipynb

scenarios-labeled.jsonl

scenarios-risked.jsonl

datasets

AMT Risk Input File Creator.ipynb

amt-risk-input.csv

app-url-dictionary.json

Dataset Utilities.ipynb

deployed-id-split.json

Scenario-Level Dataset Preparation....

scenarios-labeled-input.jsonl

scenarios-labeled.json

scenarios-risked.json

Sentence-Level Dataset Preparation...

Summary of Authored Scenarios.ipynb

Summary of Risk Survey Results.ipynb

models

bert-finetuned-ner

checkpoint-50

checkpoint-100

checkpoint-150

checkpoint-200

BERT-based NER Example Usage.ip...

BERT-based NER Training.ipynb

CRF NER Example Usage.ipynb

CRF NER Grid Search.ipynb

CRF NER Training and Testing.ipynb

OUTLINE

TIMELINE

BERT-based NER Training.ipynb

Interrater Reliability.ipynb

BERT-based NER Example Us

coding > Interrater Reliability.ipynb > Notebook Overview > import pandas as pd

Generate + Code + Markdown | Run All Restart Clear All Outputs ...

privacy\_mac (Python 3.9.25)

```
continue

# check that each dataset contains the same scenario IDs
d1 = set(scenario_ids1) - set(scenario_ids2)
if len(d1) > 0:
    print('Missing IDs from %s: %s' % (f2, d1))
    continue
d2 = set(scenario_ids2) - set(scenario_ids1)
if len(d2) > 0:
    print('Missing IDs from %s: %s' % (f1, d2))
    continue

# compile an ordered list of codes for computing Kappa
all_codes1 = []
all_codes2 = []
for scenario_id in scenario_ids1:
    all_codes1.extend(data1[scenario_id]['codes'])
    all_codes2.extend(data2[scenario_id]['codes'])

# verify the number of codes is the same between datasets
if len(all_codes1) != len(all_codes2):
    print('Code length mismatch: %s, %s' % (f1, f2))
    continue

# compute Kappa
kappa = cohen_kappa_score(all_codes1, all_codes2)
print('Cohen\'s Kappa Round %i: %0.4f' % (round, kappa))
round += 1
```

[1] ✓ 0.7s Python

... Cohen's Kappa Round 1: 0.2971  
Cohen's Kappa Round 2: 0.5746  
Cohen's Kappa Round 3: 0.6941  
Cohen's Kappa Round 4: 0.7745

Spaces: 4 {} Cell 3 of 4



EXPLORER

SCENARIO\_RISK\_SCORES

coding

samples

Interrater Reliability.ipynb

scenarios-labeled.jsonl

scenarios-risked.jsonl

datasets

AMT Risk Input File Creator.ipynb

amt-risk-input.csv

app-url-dictionary.json

Dataset Utilities.ipynb

deployed-id-split.json

Scenario-Level Dataset Preparation....

scenarios-labeled-input.jsonl

scenarios-labeled.json

scenarios-risked.json

Sentence-Level Dataset Preparation...

Summary of Authored Scenarios.ipynb

Summary of Risk Survey Results.ipynb

models

bert-finetuned-ner

checkpoint-50

checkpoint-100

checkpoint-150

checkpoint-200

BERT-based NER Example Usage.ip...

BERT-based NER Training.ipynb

CRF NER Example Usage.ipynb

CRF NER Grid Search.ipynb

CRF NER Training and Testing.ipynb

OUTLINE

TIMELINE

scenario\_risk\_scores

1

based NER Training.ipynb M

Interrater Reliability.ipynb M

AMT Risk Input File Creator.ipynb M

Generate

Code

Markdown

Run All

Restart

Clear All Outputs

privacy\_mac (Python 3.9.25)

datasets > AMT Risk Input File Creator.ipynb > rows[0]

print('\nDistribution of labeled information types:\n')

print(['%s: %s' % (c, counter[c]) for c in sorted(counter.keys())])

if len(rows) < 100:

sampled\_rows = random.sample(rows, 100)

else:

sampled\_rows = rows

filename = 'amt-risk-input.csv'

with open(filename, 'w') as f:

writer = csv.writer(f)

writer.writerow(['scenario\_id', 'app\_url', 'app\_name', 'scenario\_text', 'infotypes', 'risk\_scores'])

writer.writerows(sampled\_rows)

print('\nWrote %i rows in file: %s' % (len(sampled\_rows), filename))

[2] ✓ 0.0s Python

...

Distribution of labeled information types:

['0: 198']

Wrote 198 rows in file: amt-risk-input.csv

(variable) rows: list

rows[0]

[3] ✓ 0.0s Python

...

['MAS-R-0',

'https://play.google.com/store/apps/details?id=com.skype.raider',

'Skype',

"I use this screen to talk with my friend on Skype from the phone. I type messages to him and as long as be

''

'']

Cell 3 of 6

scenario\_risk\_scores

1

EXPLORER

SCENARIO\_RISK\_SCORES

coding

samples

Interrater Reliability.ipynb

scenarios-labeled.jsonl

scenarios-risked.jsonl

datasets

AMT Risk Input File Creator.ipynb

amt-risk-input.csv

app-url-dictionary.json

Dataset Utilities.ipynb

deployed-id-split.json

Scenario-Level Dataset Preparation...

scenarios-labeled-input.jsonl

scenarios-labeled.json

scenarios-risked.json

Sentence-Level Dataset Preparation...

Summary of Authored Scenarios.ipynb

Summary of Risk Survey Results.ipynb

models

bert-finetuned-ner

checkpoint-50

checkpoint-100

checkpoint-150

checkpoint-200

BERT-based NER Example Usage.ip...

BERT-based NER Training.ipynb

CRF NER Example Usage.ipynb

CRF NER Grid Search.ipynb

CRF NER Training and Testing.ipynb

OUTLINE

TIMELINE

mple Usage.ipynb M

CRF NER Training and Testing.ipynb M

CRF NER Example Usage.ipynb M

models > CRF NER Example Usage.ipynb > import nltk

Generate + Code + Markdown | Run All Restart Clear All Outputs ...

privacy\_mac (Python 3.9.25)

y\_pred = model.predict(X)

[5] ✓ 0.0s Python

entities = []

for i in range(len(segments)):

for j in range(len(segments[i])):

text = segments[i][j][0]

text\_start = segments[i][j][2]

text\_end = text\_start + len(text)

label = y\_pred[i][j]

print('%s\t%s' % (text.ljust(15), label))

if label.startswith('B-'):

entities.append({'start': text\_start, 'end': text\_end, 'entity\_group': label[2:]})

elif label.startswith('I-'):

entities[-1]['end'] = text\_end

[6] ✓ 0.0s Python

...

From 0

this 0

screen 0

,

I 0

like 0

to 0

search 0

for 0

anything B-COM

from I-COM

recipes I-COM

,

to 0

home B-SIM

decor I-SIM

main\*

0

4

Cell 3 of 9



scenario\_risk\_scores

1

EXPLORER

SCENARIO\_RISK\_SCORES

datasets

app-url-dictionary.json

Dataset Utilities.ipynb

deployed-id-split.json

Scenario-Level Dataset Preparation....

scenarios-labeled-input.jsonl

scenarios-labeled.json

scenarios-risked.json

Sentence-Level Dataset Preparation...

Summary of Authored Scenarios.ipynb

Summary of Risk Survey Results.ipynb

models

bert-finetuned-ner

checkpoint-50

checkpoint-100

checkpoint-150

checkpoint-200

BERT-based NER Example Usage.ip...

BERT-based NER Training.ipynb

CRF NER Example Usage.ipynb

CRF NER Grid Search.ipynb

CRF NER Training and Testing.ipynb

crf.model

information\_types.csv

Prediction Post-Processing.ipynb

scenarios-training-0.json

scenarios-training-1.json

scenarios-training-2.json

scenarios-training-3.json

OUTLINE

TIMELINE

BERT-based NER Training.ipynb

CRF NER Training and Testing.ipynb

aining and Testing.ipynb > # https://sklearn-crfsuite.readthedocs.io/en/latest/tutorial.html#let-s-use-conll-2002-data-to-build-a-ner-...

Generate + Code + Markdown | Run All Restart Clear All Outputs ... privacy\_mac (Python 3.9.25)

[3]

✓ 3.4s

Python

[4]

✓ 0.0s

Python

...

precision recall f1-score support

B-COM 0.23 0.18 0.20 39

B-QUE 0.86 0.53 0.66 79

B-SIM 0.70 0.62 0.66 626

I-COM 0.25 0.24 0.25 177

I-QUE 0.76 0.52 0.62 462

I-SIM 0.49 0.35 0.40 107

0 0.92 0.96 0.94 7332

accuracy 0.88 8822

macro avg 0.60 0.49 0.53 8822

weighted avg 0.88 0.88 0.88 8822

14

# Problems

- Messy Dependencies
- Chaotic File Structure
- Low Training Efficiency
- Jupyter Notebook is not suitable for large-scale training



# IMPROVEMENT

- Rewrite requirements.txt
- Refactor some sections
- Introduce CUDA and MPS for model training
- Consider refactoring the whole project to .py files



```
import json

data = json.load(open('scenarios-risked.json', 'r'))

# 要排除的ID数字
exclude = [45, 65, 110, 112, 121]

updated = []
for item in data:
    # 从 scenario_id 中提取数字部分 (例如 "MAS-R-45" -> 45)
    scenario_num = int(item['scenario_id'].split('-')[-1])

    if scenario_num in exclude:
        continue

    # 数据已经有 labels 字段，直接保留
    updated.append(item)

json.dump(updated, open('scenarios-risked1.json', 'w'), indent=2)

print(f"原始记录数: {len(data)}")
print(f"排除后记录数: {len(updated)}")
```

[2]



0.0s

Python

...

原始记录数: 198

排除后记录数: 193

```

from transformers import AutoTokenizer, AutoModelForTokenClassification
from transformers import pipeline
import os

# 1. 转化为绝对路径，消除歧义
model_path = os.path.abspath('./bert-finetuned-ner/checkpoint-200')

# 2. 检查一下路径是否存在（防止刚才环境重造时路径丢了）
if not os.path.exists(model_path):
    print(f"❌ 警告：路径不存在 {model_path}，请确认文件夹名是否正确！")
else:
    # 3. 实例化
    tokenizer = AutoTokenizer.from_pretrained(model_path)
    model = AutoModelForTokenClassification.from_pretrained(model_path, ignore_mismatched_sizes=True)
    device = "mps" if torch.backends.mps.is_available() else "cuda" if torch.cuda.is_available() else "cpu"

    nlp = pipeline('ner', model=model, tokenizer=tokenizer, aggregation_strategy='first', device = torch.device(device))
    print("✅ Pipeline 加载成功！")

```

Pythor

```

# train the model
args = TrainingArguments(
    "bert-finetuned-ner",
    eval_strategy = "epoch",
    save_strategy = "epoch",
    learning_rate = 2e-5,
    num_train_epochs = 4,
    weight_decay = 0.01,
    push_to_hub = False,
    use_mps_device = True if torch.backends.mps.is_available() else False,
    cuda_device = 0 if torch.cuda.is_available() else -1
)

```



requirements.txt

```
1 # --- 核心深度学习与 Transformer 框架 ---
2
3 torch>=2.0.1
4 transformers>=4.30.2
5 sentencepiece>=0.1.99
6 safetensors>=0.3.1
7 tokenizers>=0.13.3
8
9 # --- 传统机器学习与 CRF 模块 ---
10
11 scikit-learn>=1.2.2
12 python-crfsuite>=0.9.9
13 sklearn-crfsuite>=0.3.6
14
15 # --- 自然语言处理（NLP）基础工具 ---
16
17 nltk>=3.8.1
18 spacy>=3.5.3
19
20
21 # --- 科学计算与数据处理 ---
22
23 numpy>=1.22,<1.24
24 pandas>=2.0.2
25 scipy>=1.10.1
26 matplotlib>=3.7.1
27 h5py>=3.9.0
28
29 # --- 训练监控与评估 ---
30
31 seqeval>=1.2.2
32 evaluate>=0.4.0
33 datasets>=2.13.0
34 tqdm>=4.65.0
35 tensorboard>=2.12.3
36
37 # --- 交互式环境与内核 ---
38
```

```
# --- 深度学习与 Transformer 框架 ---
torch>=2.0.1
transformers>=4.30.2
sentencepiece>=0.1.99
safetensors>=0.3.1
tokenizers>=0.13.3

# --- 传统机器学习与 CRF 模块 ---
scikit-learn>=1.2.2
python-crfsuite>=0.9.9
sklearn-crfsuite>=0.3.6

# --- 自然语言处理（NLP）基础工具 ---
nltk>=3.8.1
spacy>=3.5.3

# --- 科学计算与数据处理 ---
numpy>=1.22,<1.24
pandas>=2.0.2
scipy>=1.10.1
matplotlib>=3.7.1
h5py>=3.9.0

# --- 训练监控与评估 ---
seqeval>=1.2.2
evaluate>=0.4.0
datasets>=2.13.0
tqdm>=4.65.0
tensorboard>=2.12.3

# --- 交互式环境与内核 ---
```

THANKS FOR LISTENING